

如何让 AI 与 Figma 更好地结合

本文作者：焦健，TRAE技术专家

为什么 AI 总是没法还原设计稿？

长期以来，将设计稿无损、高效地转化为高质量前端代码，始终是产研团队面临的核心挑战。尽管近年来 AI 代码生成工具（AI Coding）层出不穷，但多数实践依然停留在“可用但不好用”的阶段。究其原因，主要有以下几点：

- 设计信息的“有损压缩”**：Figma 的设计文件本质上是复杂的矢量图形数据。早期的 Design-to-Code (D2C) 工具或 AI 插件，为了让模型能够“理解”设计，往往需要对原始设计信息进行大幅简化和压缩，例如仅提取图层的基本属性（位置、尺寸、颜色）。这个过程不可避免地会丢失大量关键信息，如自适应布局规则（Auto Layout）、组件的变体状态、交互逻辑以及最重要的——设计意图。模型拿到的是一份“残缺”的上下文，自然无法 100% 还原视觉稿。
- 上下文的挤占与注意力的涣散**：AI Coding 工具在生成代码时，其上下文窗口是有限的。当开发者在 IDE 中编码时，窗口中充斥着当前代码库的结构、依赖、规范等信息。此时，如果再强行注入大量、未经提炼的设计稿信息，很容易挤占宝贵的上下文空间，导致模型注意力涣散，无法精准聚焦于当前任务，甚至产生“代码幻觉”。
- 缺乏对组件的真实认知**：现代前端开发高度依赖组件化。然而，AI 对设计稿中的“组件”理解，往往停留在视觉层面——它看到了一个“看起来像按钮”的矩形，而不是一个链接到代码库中 `Button` 组件的实例。这导致 AI 倾向于重新生成一个视觉相似但毫无工程价值的 `<div>` 结构，而非复用既有的、经过验证的、包含完整逻辑与样式的真实组件。
- 整页生成的信息爆炸**：实践证明，直接将一整个复杂页面的设计稿链接扔给 AI，期望它能一键生成完美代码，几乎是不可能完成的任务。页面越复杂，信息量越大，AI 的还原准确度就越低，最终产出的代码往往难以维护，重构成本甚至高于手写。

 **问题的根源：D2C 的瓶颈**一方面 在于模型本身的上下文控制，另一方面在于**输入侧设计上下文的质量与精确度**。模型需要的是“信噪比”极高的、结构化的、与真实代码世界关联的设计意图，而非庞杂、离散的视觉描述。

Figma 原始数据非常庞大，所以之前市面上的 Figma MCP 都做了**有损压缩**，UI 信息必然会丢失，理论上是无法 100% 还原视觉稿的。实现细节可以参考 早先很热门的 [Figma-Context-MCP](#) 的 DeepWiki >>

即使 Figma 官方推出的 MCP ，也是通过返回 从设计稿生成的 React+Tailwind 的代码 配合着节点元信息，来尽可能对信息进行“压缩”。

我们如何让结果能更符合期望？

总的来说，优化策略可以归结为两大方向：

- 约束 AI 的实现规模
- 提供更高质量的上下文

1. 选择合适的模型与输入方式

工欲善其事，必先利其器。选择正确的模型和输入方式，是提升 D2C 效率的第一步。

- **优先选择长上下文模型**：处理复杂设计稿时，上下文窗口的大小至关重要。像 `Doubao-Seed-Code` (184k) 这样窗口较小的模型，在面对大量设计信息时容易捉襟见肘。因此，推荐优先选择如 **Claude Opus** 或 **Gemini 3 Pro** 等具备更长上下文窗口（200k）的模型。
 - GPT-5 拥有最大的上下文窗口 272K ，但实际使用体验下来受限于模型训练侧重和注意力，感觉额外的 72k 上下文并没有产生明显帮助。
- **利用多模态能力，截图辅助理解**：纯粹的结构化数据有时不足以传达完整的布局意图。在向 AI 提问时，附上一张设计稿的**截图**，可以极大地帮助模型理解整体布局和视觉关系，防止生成结果出现严重跑偏。



2. 模块化拆分 > 整页生成



不要期望“一键完美”：所有第三方工具都无法完全替代前端工程师。它们的核心价值在于**加速从设计到代码的初稿阶段**，而非终点。

实践证明，直接提供整页的复杂设计 Figma 链接进行代码生成的效果不佳。最佳实践是**将页面拆分为独立的模块像搭积木一样搭建**，分多次请求 AI，逐个模块进行确认和还原，这样是最能平衡工作量和可用性的。通过对每个组件能显著提升代码质量和还原准确度。

整页生成的诱惑与陷阱

市面上很多 AI Design 产品“一句话生成 App”的能力非常诱人。然而，直接用于生产环境时，其局限性也非常明显：

- 缺乏深层逻辑理解：**AI 可以生成漂亮的界面和流畅的转场动画，但很难精准理解复杂的业务逻辑、状态管理和数据流。生成的代码往往是“所见即所得”的 UI 快照，缺乏健壮的工程架构。
- 组件系统的无知：**即使 AI 能够调用组件库，它也可能因为对上下文理解的偏差而误用组件，或是在需要复用组件的地方重新创建了相似的 UI。
- 维护性与性能问题：**AI 生成的代码可能存在冗余、性能瓶颈或不符合团队编码规范的问题。在复杂的、需要长期迭代的项目中，直接使用这些代码可能会埋下技术债务。

何时用 AI，何时靠人？

与其追求完全自动化，更现实的策略是建立一个人机协同的工作流，在不同阶段发挥各自的优势。

场景/阶段	推荐使用 AI 生成	必须人工/半自动微调
0 → 1 创意验证	- 使用 Figma Make 或类似工具，快速将产品想法转化为可交互原型，用于内部评审或早期用户访谈。 - 快速生成多个不同风格的视觉方案。	人工：梳理核心用户流程和信息架构，确保原型逻辑自治。
1 → N 模块化开发	(推荐) 将页面拆分为独立的、功能内聚的模块（如搜索栏、卡片列表、表单），逐个模块让 AI 生成代码。 - AI 尤其擅长生成结构固定、样式重复的静态展示类模块。	人工：定义模块的接口 (Props) 和状态，确保其可复用性和可测试性。 半自动：AI 生成初版代码后，由工程师审查并重构，使其符合项目编码规范、集成真实数据接口、并应用设计系统中的真实组件和Token。
复杂交互与可视化	不建议直接使用 AI 生成复杂的图表、地图、动画或需要精细手势操作的组件。	人工：由前端工程师手写或使用专业库（如 ECharts, D3.js, Lottie）实现，确保性能、兼容性和可定制性。

设计系统组件开发	可以让 AI 生成组件的初始结构（如 <code>Button.tsx</code> 的基本 JSX 和样式），作为开发的起点。	人工：必须由工程师精细打磨，编写完整的 props 定义、事件处理、状态逻辑、测试用例和文档，确保其作为设计系统资产的质量。
----------	--	---

3. 设计规范是前提

无论使用何种工具，其输出质量都与输入的设计稿质量直接相关。一个没有使用 Auto Layout、组件化和 Variables 的“自由”设计稿，任何工具都难以生成高质量的代码。

4. 更好地获得设计稿数据

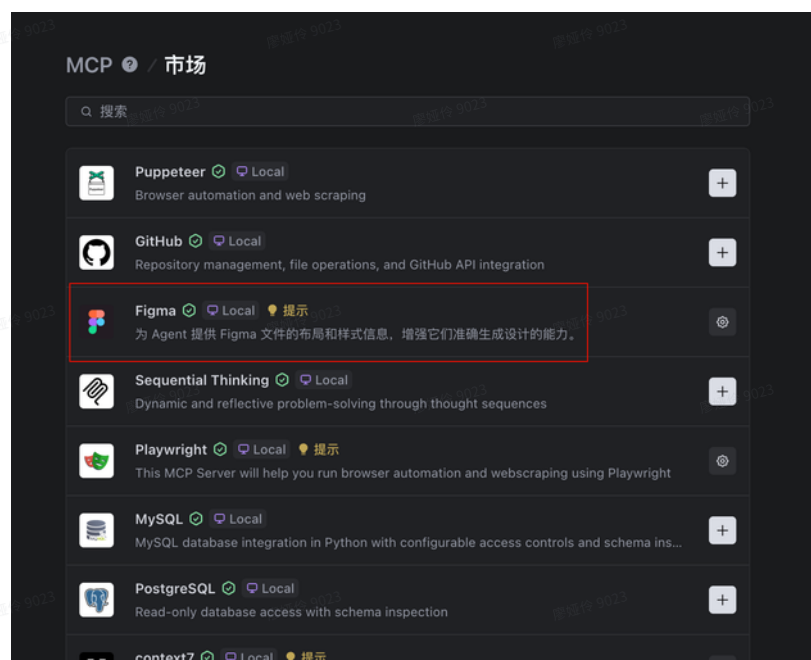
 这部分选择对了，体验提升会非常大。

方法	核心优势	核心劣势	推荐指数
Figma 官方 MCP + Code Connect + SKILL	工程质量最高，能复用真实组件	配置较复杂，需本地运行 Figma	★★★★★
TRAE Figma	操作最便捷，与开发环境无缝集成	无法复用组件和 Design Token	★★★
自定义 OpenAPI 脚本生成 IR	灵活性最高，可深度定制	心智负担重，中间产物过大	★★
市场原本 MCP	无	过于简易	完全不推荐

市场中的 Figma MCP

TRAE 市场中的 Figma MCP，内里只有两个简易的 tool，获取的设计稿也只是简单的 metadata，相对失真。因而不推荐使用。

- `get_figma_data` 获取设计文件的数据
- `download_figma_images` 调用Figma API下载 svg 等图片信息



取而代之的，我们有几种方式来获取更多的有效上下文：

4.1 使用 Figma App Local MCP + SKILL

Figma 官方提供了 MCP，需要本地开启 Figma App 时使用

<https://developers.figma.com/docs/figma-mcp-server/local-server-installation/>

工具名称	功能描述
get_design_context	获取图层或选区的设计上下文,React+Tailwind,Vue,Plain HTML+CSS
get_variable_defs	返回 Figma 选区中使用的变量和样式
get_code_connect_map	获取 Figma 节点 ID 与代码库中对应代码组件之间的映射关系
add_code_connect_map	添加 Figma 节点 ID 与代码库中对应代码组件之间的映射关系
get_screenshot	允许代理对选区进行截图
create_design_system_rules	创建规则文件，为代理提供正确的上下文以将设计转换为前端代码
get_metadata	返回选区的稀疏 XML 表示，包含图层 ID、名称、类型、位置和尺寸等基本属性
get_figjam	将 FigJam 图表（如应用架构工作流）转换为 XML
whoami (仅远程)	返回已认证到 Figma 的用户身份信息
get_code_connect_suggestions	Figma 触发的工具调用，用于查找 Figma 节点 ID 与代码库中对应代码组件的 Code Connect 映射建议

send_code_connect_ma
ppings

在调用 get_code_connect_suggestions 后使用的 Figma 触发工具，用于确认建议的 Code Connect 映射

<https://developers.figma.com/docs/figma-mcp-server/tools-and-prompts/>

配合SKILL，约束其行为

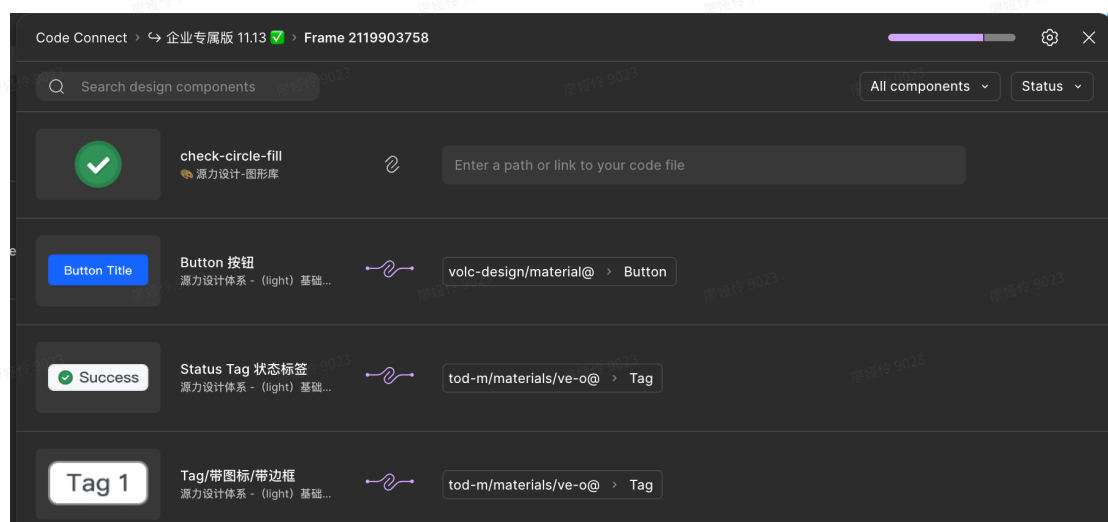
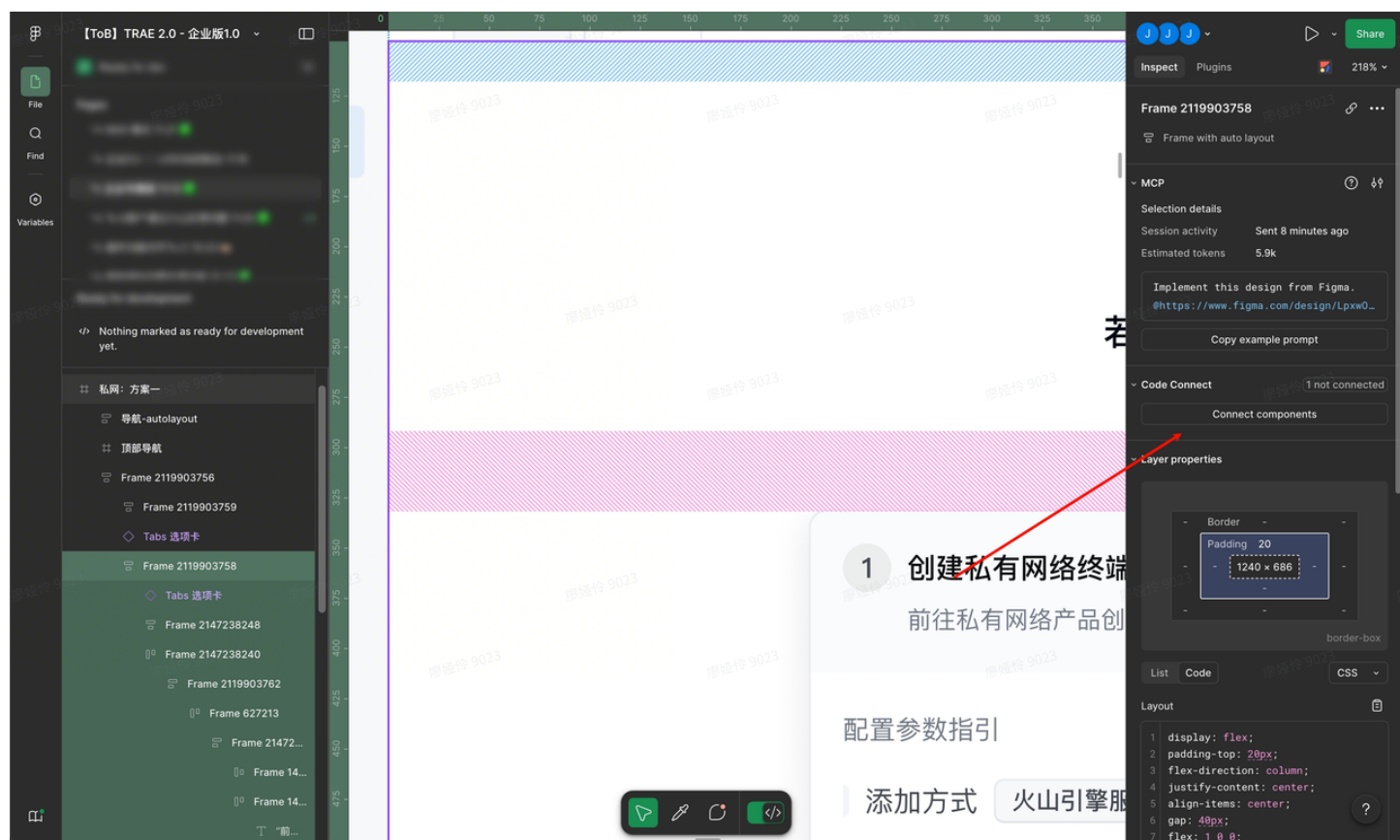


 SKILL.md

Figma Code Connect

使用 Figma 官方 MCP，可以使用 Figma 的 Code Connect 功能，将代码仓库内的组件文件和设计稿内的组件进行关联，达到进一步提供精准还原信息的作用。

Code Connect 允许你将 Figma 中的组件（或其变体）直接映射到代码库中对应的组件文件（如 React、Vue 组件）。当 AI 通过 MCP 读取设计上下文时，它不再是猜测，而是**明确地知道**这个设计元素应该使用哪个代码组件来渲染。



它的作用有两点：

- **提供高质量的上下文：**当 AI Coding 工具通过 Figma 的 MCP Server 请求设计上下文时，Code Connect 的映射关系会作为核心信息被包含进去。AI 会收到类似这样的指令：“这里是一个 `Button` 组件，它的代码在 `@/components/ui/button.tsx`，其中 `type` 属性是 `Primary`，`size` 属性是 `Default`”。
- **从“生成”到“复用”：**基于这样的精确上下文，AI 的任务从“重新发明一个按钮”转变为“正确地调用和配置一个已有的 `Button` 组件”。这从根本上保证了代码的质量、一致性和可维护性。如下方示例所示，未使用 Code Connect 时，AI 只能生成一个普通的 `div`；而使用后，则会直接 `import` 真实的 `Button` 组件

代码块

```

1 // 未使用 Code Connect 的 AI 输出 (示意)
2 const OldComponent = () => {
3   return (
4     <div style={{ padding: '8px 16px', backgroundColor: '#1664FF', color:
      'white', borderRadius: '4px' }}>
5       这是一个按钮
6     </div>
7   );
8 }
9
10 // 使用 Code Connect 后的 AI 输出 (示意)
11 import Button from "@components/ui/button.tsx"
12 const ComponentCodeConnect = ()=>{
13   return (
14     <CodeConnectSnippet data-node-id="7218:16632" data-name="Button 按钮"
      data-annotations="点击跳转至: [https://console.volcengine.com/pl/region:pl+cn-
      beijing/endpoints/create?endpointType=endpoint]
      (https://console.volcengine.com/pl/region:pl+cn-beijing/endpoints/create?
      endpointType=endpoint)">
15       <Button prefixIcon前置图标="517:20881" suffixIcon后置图标
        ="630:34396" suffixIcon后置图标={false} prefixIcon前置图标={false} type类型
        ="Primary 主要" theme主题="Default 默认" size尺寸="Default 32" state状态="Default
        默认" iconOnly仅图标="False" disable禁用="False" language="CN" />
16     </CodeConnectSnippet>
17   )
18 }

```

觉得手动链接麻烦？Figma Local MCP 里提供了 `add_code_connect_map` 功能，你可以利用它，读取设计稿后让模型替你寻找仓库中的组件代码，让 AI 辅助完成组件的映射工作，并最终由你进行审核确认。

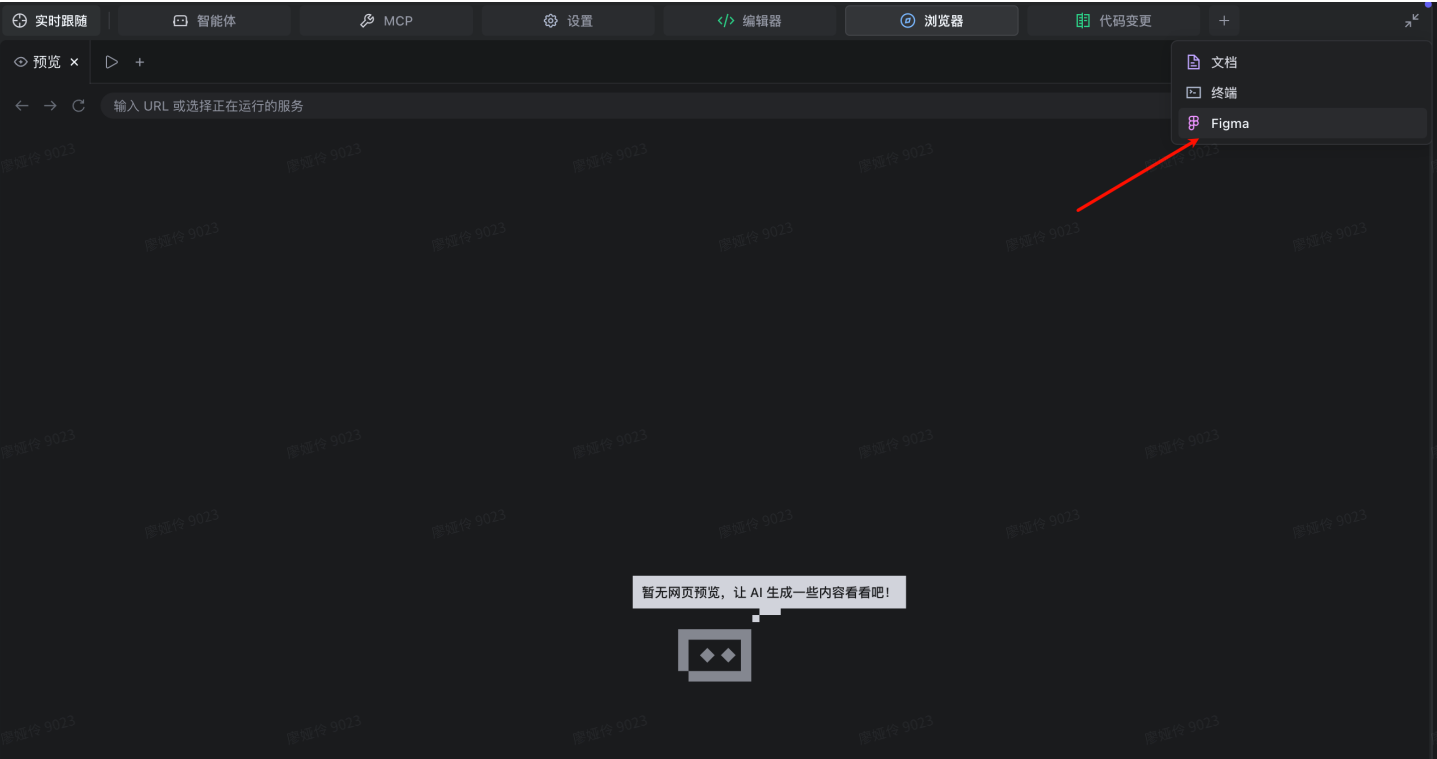
4.2 TRAE Figma

工作原理：在 TRAE IDE 中通过插件直接与 Figma 联动。在 Figma 的 Dev Mode 下，框选一个图层或组件，点击“添加到对话”，插件会自动抓取该选区的信息（通常是一张截图 + 生成的 JSX/CSS 代码）并注入到 AI 对话中。

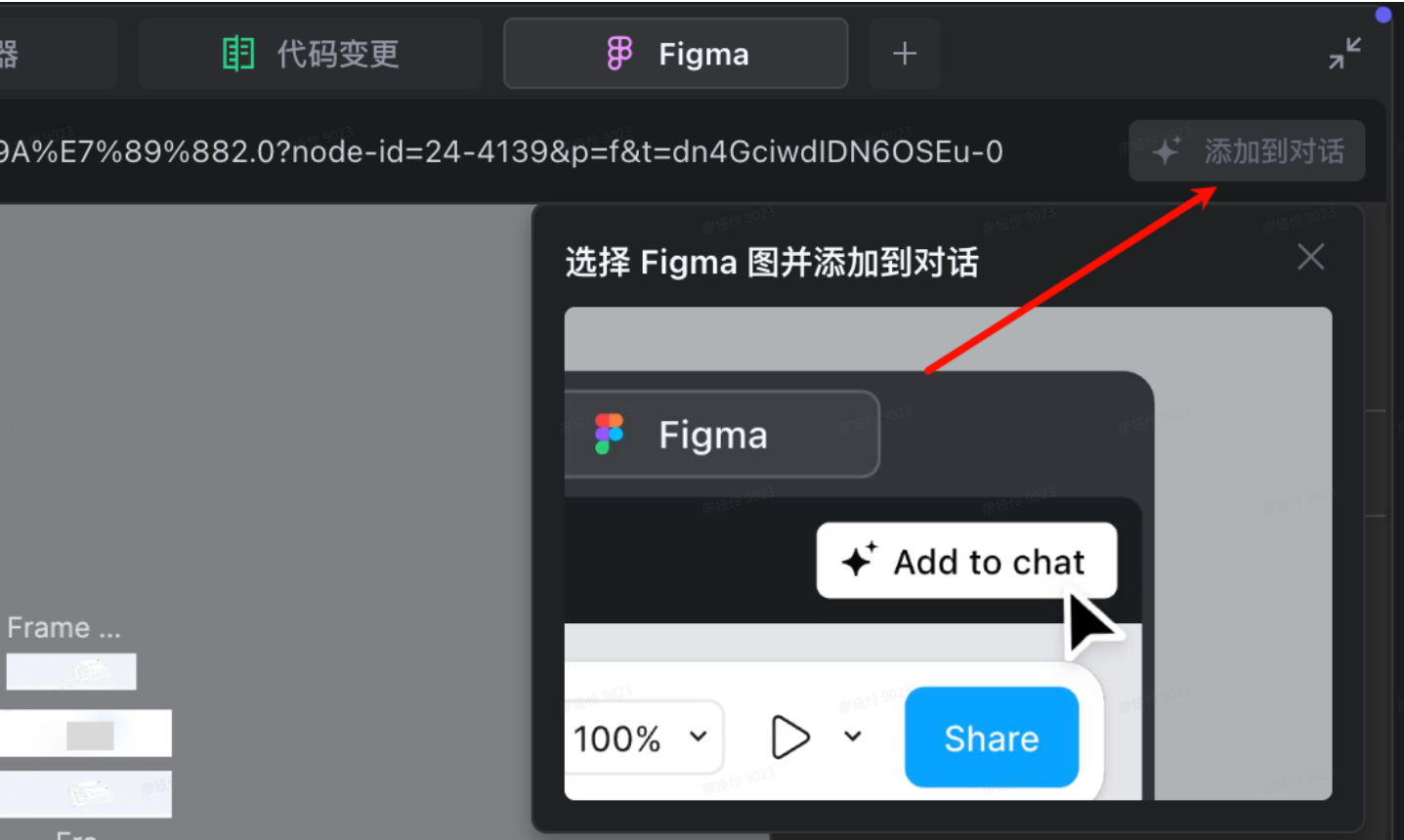
 **特点：**AI 能够准确获取文案内容，但由于缺乏 Code Connect 和 Design Token 信息，所有元素都是由 AI 重新生成的 `div` 结构，无法复用现有组件库。这与“像素工匠”模式类似，维护成本较高。

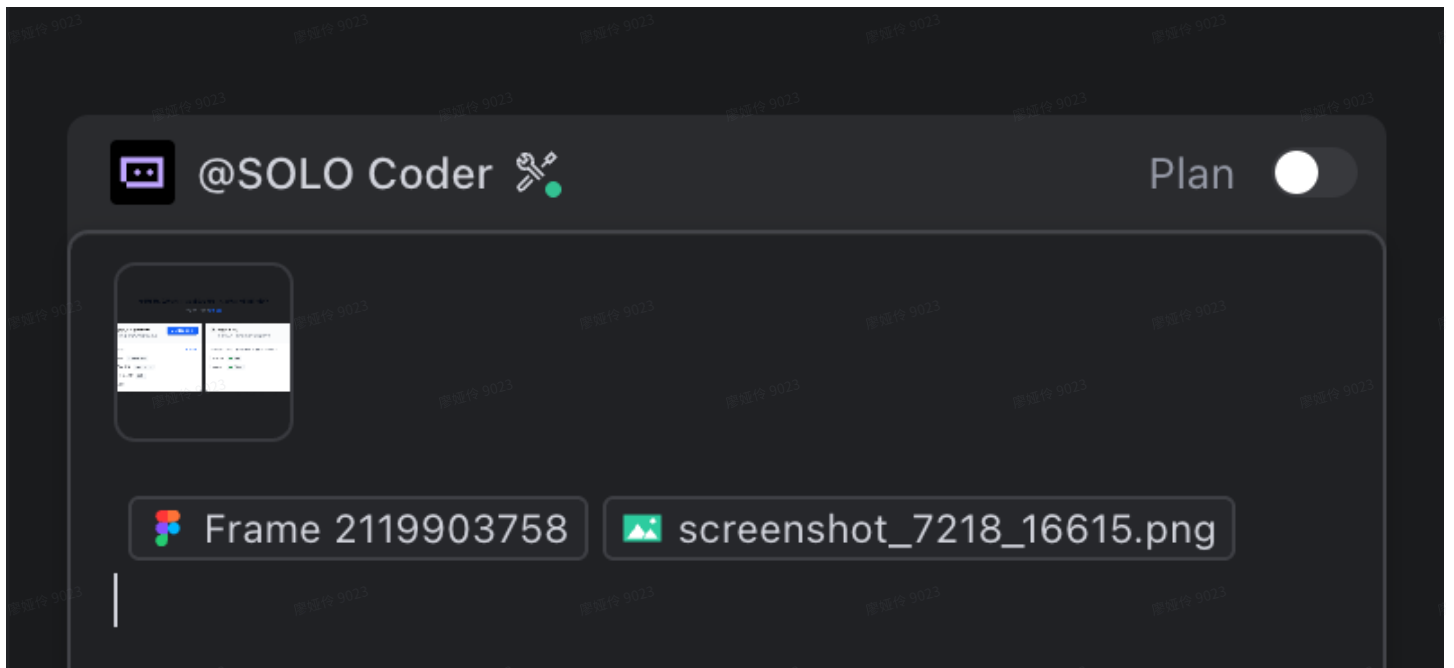
具体操作：

打开 TRAE IDE 在顶栏的 “+” 中点击可以找到。



在开启 Figma Dev Mode 之后，我们可以直接将选区一键添加到Solo中





点击添加到对话后，TRAE 会在项目的 `.figma` 文件夹中生成以 `nodeid` 为名的文件夹，存放生成产物，一般是一个 `tsx` 文件、一个 `less` 文件以及资源文件，以及一张设计稿整体截图。

TRAE Figma 提供给模型的 `tsx` 文件

```
1  import React from 'react';
2
3  import styles from './index.module.scss';
4
5  const Component = () => {
6    return (
7      <div className={styles.frame2119903758}>
8        <div className={styles.frame2147238248}>
9          <p className={styles.text}>
10            若需要通过私网访问 TRAE 企业专属版，可按照如下步骤进行操作
11          </p>
12          <p className={styles.text4}>
13            <span className={styles.text2}>了解更多，可查看</span>
14            <span className={styles.text3}>配置详情</span>
15          </p>
16        </div>
17      <div className={styles.frame2147238240}>
18        <div className={styles.frame2119903762}>
19          <div className={styles.frame627213}>
20            <div className={styles.frame2147238243}>
21              <div className={styles.frame1410097627}>
22                <div className={styles.frame1000002703}>
23                  <p className={styles.a1}>1</p>
24                </div>
25                <p className={styles.text5}>创建私有网络终端节点</p>
26              </div>
```

```
<p className={styles.text6}>前往私有网络产品创建终端节点</p>
</div>
<div className={styles.instance}>
  <p className={styles.buttonTitle}>去创建终端节点</p>
</div>
</div>
<div className={styles.frame1000002806}>
  <div className={styles.frame194388}>
    <p className={styles.text7}>配置参数指引</p>
    <p className={styles.text8}>查看示例</p>
  </div>
  <div className={styles.frame1943882}>
    
    <p className={styles.text9}>添加方式</p>
    <div className={styles.instance2}>
      <p className={styles.tag1}>火山引擎服务    </p>
    </div>
  </div>
  <div className={styles.frame1943883}>
    
    <p className={styles.text9}>终端节点服务 </p>
    <div className={styles.instance2}>
      <p className={styles.tag1}>com.xxx.xxx    </p>
    </div>
  </div>
  <div className={styles.frame1943884}>
    
    <p className={styles.text9}>私有 DNS 名称</p>
    <div className={styles.instance3}>
      <p className={styles.tag12}>启用</p>
    </div>
  </div>
  <div className={styles.frame1943885}>
    
    <p className={styles.text9}>完成购买</p>
  </div>
</div>
```

```
74     </div>
75   </div>
76 </div>
77 <div className={styles.frame2119903764}>
78   <div className={styles.frame6272132}>
79     <div className={styles.frame627147}>
80       <div className={styles.frame1000002703}>
81         <p className={styles.a1}>2</p>
82       </div>
83       <p className={styles.text10}>检查可用状态</p>
84     </div>
85     <p className={styles.text6}>购买完成后，根据列表状态判断是否可用</p>
86   </div>
87   <div className={styles.frame1000002807}>
88     <p className={styles.text11}>
89       检查列表，满足以下条件则表示【私有网络可用】
90     </p>
91     <div className={styles.frame1943886}>
92       
96       <p className={styles.text12}>实例状态</p>
97       <div className={styles.instance4}>
98         
102        <p className={styles.tag12}>可用</p>
103      </div>
104    </div>
105    <div className={styles.frame1943887}>
106      
110      <p className={styles.text12}>连接状态</p>
111      <div className={styles.instance4}>
112        
116        <p className={styles.tag12}>已连接</p>
117      </div>
118    </div>
119  </div>
120</div>
```

```
121     </div>
122   </div>
123 );
124 }
125
126 export default Component;
127
```

TRAE Figma 提供给模型的 LESS 文件

```
1  * {
2    box-sizing: border-box;
3  }
4
5  .frame2119903758 {
6    display: flex;
7    flex-direction: column;
8    align-items: center;
9    justify-content: center;
10   padding-top: 20px;
11   width: 1240px;
12   height: 706px;
13   row-gap: 40px;
14
15   .frame2147238248 {
16     display: inline-flex;
17     flex-direction: column;
18     flex-shrink: 0;
19     align-items: center;
20     align-self: stretch;
21     margin-right: 356px;
22     margin-left: 356px;
23     row-gap: 16px;
24
25     .text {
26       flex-shrink: 0;
27       line-height: 26px;
28       letter-spacing: 0.05px;
29       color: var(--Fill-color-fill-4, #020814);
30       font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
31       SimHei, Arial, Helvetica, sans-serif;
32       font-size: 18px;
33       font-weight: 500;
34     }
35
36     .text4 {
```

```
36     flex-shrink: 0;
37     line-height: 20px;
38     letter-spacing: 0.04px;
39     color: #000000;
40     font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
SimHei, Arial, Helvetica, sans-serif;
41     font-size: 12px;
```

```
42
43     .text2 {
44         line-height: 20px;
45         letter-spacing: 0.04px;
46         color: var(--Text-color-text-3, #737a87);
47         font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
SimHei, Arial, Helvetica, sans-serif;
48         font-size: 12px;
49     }
```

```
50
51     .text3 {
52         line-height: 20px;
53         letter-spacing: 0.04px;
54         color: var(--Primary-Color-primary-6, #1664ff);
55         font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
SimHei, Arial, Helvetica, sans-serif;
56         font-size: 12px;
57     }
58 }
59 }
```

```
60
61     .frame2147238240 {
62         display: flex;
63         flex-shrink: 0;
64         align-items: flex-start;
65         justify-content: center;
66         column-gap: 20px;
67         padding-bottom: 80px;
68         width: 1161px;
```

```
69
70     .tag12 {
71         flex-shrink: 0;
72         line-height: 20px;
73         letter-spacing: 0.04px;
74         color: var(--Text-color-text-2, #42464e);
75         font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
SimHei, Arial, Helvetica, sans-serif;
76         font-size: 12px;
77         font-weight: 500;
78     }
```



```
79
80 .rectangle34624872 {
81     flex-shrink: 0;
82     width: 3px;
83     height: 16px;
84 }
85
86 .text6 {
87     flex-shrink: 0;
88     align-self: stretch;
89     padding: 0px 0px 0px 32px;
90     line-height: 20px;
91     letter-spacing: 0.04px;
92     color: var(--Text-color-text-3, #737a87);
93     font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
SimHei, Arial, Helvetica, sans-serif;
94     font-size: 12px;
95 }
96
97 .frame10000002703 {
98     display: inline-flex;
99     flex-direction: column;
100     flex-shrink: 0;
101     align-items: center;
102     justify-content: center;
103     border-radius: 40px;
104     background: var(--Grey-grey-3, #ecedea);
105     padding: 1px 2px;
106     row-gap: 10px;
107
108     .a1 {
109         flex-shrink: 0;
110         width: 20px;
111         text-align: center;
112         line-height: 22px;
113         letter-spacing: 0;
114         color: var(--Text-color-text-2, #42464e);
115         font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
SimHei, Arial, Helvetica, sans-serif;
116         font-size: 14px;
117         font-weight: 500;
118     }
119 }
120
121 .frame2119903762 {
122     display: flex;
123     flex-direction: column;
```

```
124 flex-shrink: 0;
125 align-items: flex-start;
126 outline-width: 1px;
127 outline-style: solid;
128 outline-color: var(--Line-color-border-2, #eaeef1);
129 border-radius: 8px;
130 box-shadow: 0px 5px 20px 0px #0000001a;
131 background: var(--Background-color-bg-1, #ffffff);
```

```
132
133 .frame627213 {
134   display: flex;
135   flex-shrink: 0;
136   align-items: flex-start;
137   align-self: stretch;
138   column-gap: 4px;
139   border-radius: 8px 8px 0px 0px;
140   background: var(--Background-hover-color-bg-3, #fafbfc);
141   padding: 16px;
```

```
142
143 .frame2147238243 {
144   display: flex;
145   flex-direction: column;
146   flex-grow: 1;
147   align-items: flex-start;
148   row-gap: 4px;
```

```
149
150 .frame1410097627 {
151   display: inline-flex;
152   flex-shrink: 0;
153   align-items: center;
154   align-self: stretch;
155   column-gap: 8px;
156   margin-right: 67px;
```

```
157
158 .text5 {
159   flex-shrink: 0;
160   padding: 1px 0px;
161   line-height: 22px;
162   letter-spacing: 0.04px;
163   color: var(--Text-color-text-1, #0c0d0e);
164   font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft
YaHei", SimHei, Arial, Helvetica, sans-serif;
165   font-size: 14px;
166   font-weight: 500;
167 }
168 }
169 }
```

```
170
171 .instance {
172     display: inline-flex;
173     flex-shrink: 0;
174     align-items: center;
175     justify-content: center;
176     column-gap: 8px;
177     border-radius: 4px;
178     box-shadow: var(
179         --shadow-Assembly-Button-Blue-Default,
180         0px 0px 0px 1px #1759da,
181         0px 2px 1px 0px #00000026
182     );
183     background: var(--Primary-Color-primary-6, #1664ff);
184     padding: 5px 16px;
185     overflow: hidden;
```

```
186
187 .buttonTitle {
188     flex-shrink: 0;
189     line-height: 22px;
190     letter-spacing: 0.04px;
191     color: var(--Text-color-white, #ffffff);
192     font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
SimHei, Arial, Helvetica, sans-serif;
193     font-size: 13px;
194     font-weight: 500;
195 }
196 }
197 }
```

```
198
199 .frame10000002806 {
200     display: flex;
201     flex-direction: column;
202     flex-shrink: 0;
203     align-items: flex-start;
204     align-self: stretch;
205     border-radius: 0px 0px 8px 8px;
206     background: #ffffff;
207     padding: 12px 16px;
208     row-gap: 4px;
```

```
209
210 .text9 {
211     flex-shrink: 0;
212     line-height: 22px;
213     letter-spacing: 0.04px;
214     color: var(--Text-color-text-2, #42464e);
```

```
215     font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
SimHei, Arial, Helvetica, sans-serif;
216     font-size: 14px;
217 }
218
219 .instance2 {
220     display: inline-flex;
221     flex-shrink: 0;
222     align-items: flex-start;
223     column-gap: 10px;
224     border-radius: 4px;
225     box-shadow: var(
226         --shadow-Input-tab,
227         0px 1px 1px 0px #0000000a,
228         0px 0px 0px 1px #d5dbe380
229     );
230     background: var(--Background-2, #f6f8fa);
231     padding-right: 8px;
232     padding-left: 8px;
233     overflow: hidden;
234
235     .tag1 {
236         flex-shrink: 0;
237         line-height: 20px;
238         letter-spacing: 0.04px;
239         color: var(--Text-color-text-2, #41464f);
240         font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
SimHei, Arial, Helvetica, sans-serif;
241         font-size: 12px;
242         font-weight: 500;
243     }
244 }
245
246 .frame194388 {
247     display: flex;
248     flex-shrink: 0;
249     align-items: center;
250     align-self: stretch;
251     justify-content: space-between;
252     padding-top: 5px;
253     padding-bottom: 5px;
254     height: 32px;
255
256     .text7 {
257         flex-shrink: 0;
258         line-height: 22px;
259         letter-spacing: 0.04px;
```

```
260     color: var(--Text-color-text-3, #737a87);
261     font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
SimHei, Arial, Helvetica, sans-serif;
262     font-size: 13px;
263 }
264
265 .text8 {
266     flex-shrink: 0;
267     line-height: 20px;
268     letter-spacing: 0;
269     color: var(--B1-6-1664FF-, #1664ff);
270     font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
SimHei, Arial, Helvetica, sans-serif;
271     font-size: 12px;
272 }
273 }
274
275 .frame1943882 {
276     display: inline-flex;
277     flex-shrink: 0;
278     align-items: center;
279     align-self: stretch;
280     column-gap: 8px;
281     padding: 5px 203px 5px 0px;
282 }
283
284 .frame1943883 {
285     display: inline-flex;
286     flex-shrink: 0;
287     align-items: center;
288     align-self: stretch;
289     column-gap: 8px;
290     margin-right: 178px;
291     padding-top: 5px;
292     padding-bottom: 5px;
293 }
294
295 .frame1943884 {
296     display: inline-flex;
297     flex-shrink: 0;
298     align-items: center;
299     align-self: stretch;
300     column-gap: 8px;
301     margin-right: 213px;
302     padding-top: 5px;
303     padding-bottom: 5px;
304 }
```

```
305 .instance3 {
306   display: inline-flex;
307   flex-shrink: 0;
308   align-items: flex-start;
309   column-gap: 10px;
310   border-radius: 4px;
311   box-shadow: var(
312     --shadow-Input-tab,
313     0px 1px 1px 0px #0000000a,
314     0px 0px 0px 1px #d5dbe380
315   );
316   background: var(--Background-2, #f6f8fa);
317   padding-right: 8px;
318   padding-left: 8px;
319   overflow: hidden;
320 }
321 }
```

```
323 .frame1943885 {
324   display: inline-flex;
325   flex-shrink: 0;
326   align-items: center;
327   align-self: stretch;
328   column-gap: 8px;
329   margin-right: 300px;
330   padding-top: 5px;
331   padding-bottom: 5px;
332 }
333 }
334 }
```

```
336 .frame2119903764 {
337   display: flex;
338   flex-direction: column;
339   flex-shrink: 0;
340   align-items: flex-start;
341   outline-width: 1px;
342   outline-style: solid;
343   outline-color: var(--Line-color-border-2, #eaeef1);
344   border-radius: 8px;
345   box-shadow: 0px 5px 20px 0px #0000001a;
346   background: var(--Background-color-bg-1, #ffffff);
347   height: 280px;
348 }
```

```
349 .frame62721132 {
350   display: flex;
351   flex-direction: column;
```

```
352     flex-shrink: 0;
353     align-items: flex-start;
354     align-self: stretch;
355     border-radius: 8px 8px 0px 0px;
356     background: var(--Background-hover-color-bg-3, #fafbfc);
357     padding: 16px;
358     row-gap: 4px;
359
360     .frame627147 {
361         display: inline-flex;
362         flex-shrink: 0;
363         align-items: center;
364         align-self: stretch;
365         column-gap: 8px;
366         margin-right: 251px;
367
368         .text10 {
369             flex-shrink: 0;
370             line-height: 22px;
371             letter-spacing: 0.04px;
372             color: var(--Text-color-text-1, #0c0d0e);
373             font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
SimHei, Arial, Helvetica, sans-serif;
374             font-size: 14px;
375             font-weight: 500;
376         }
377     }
378 }
379
380 .frame1000002807 {
381     display: flex;
382     flex-direction: column;
383     flex-shrink: 0;
384     align-items: flex-start;
385     align-self: stretch;
386     border-radius: 0px 0px 8px 8px;
387     background: #ffffff;
388     padding: 12px 16px;
389     row-gap: 4px;
390
391     .text12 {
392         flex-shrink: 0;
393         line-height: 22px;
394         letter-spacing: 0.04px;
395         color: var(--Text-color-text-2, #42464e);
396         font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
SimHei, Arial, Helvetica, sans-serif;
```

```
397     font-size: 13px;
398   }
399
400   .instance4 {
401     display: inline-flex;
402     flex-shrink: 0;
403     align-items: center;
404     justify-content: center;
405     column-gap: 4px;
406     border-radius: 4px;
407     box-shadow: var(
408       --shadow-Assembly-Data-entry-default,
409       0px 0px 0px 1px #dde2e9
410     );
411     background: var(--Background-2, #f6f8fa);
412     padding-right: 6px;
413     padding-left: 6px;
414     overflow: hidden;
415
416     .checkCircleFill {
417       flex-shrink: 0;
418       width: 14px;
419       height: 14px;
420     }
421   }
422
423   .text11 {
424     flex-shrink: 0;
425     padding: 5px 0px;
426     line-height: 22px;
427     letter-spacing: 0.04px;
428     color: var(--Text-color-text-3, #737a87);
429     font-family: "PingFang SC", "Hiragino Sans GB", "Microsoft YaHei",
430     SimHei, Arial, Helvetica, sans-serif;
431     font-size: 13px;
432   }
433
434   .frame1943886 {
435     display: inline-flex;
436     flex-shrink: 0;
437     align-items: center;
438     align-self: stretch;
439     column-gap: 8px;
440     margin-right: 241px;
441     padding-top: 5px;
442     padding-bottom: 5px;
```



```
443
444     .frame1943887 {
445         display: inline-flex;
446         flex-shrink: 0;
447         align-items: center;
448         align-self: stretch;
449         column-gap: 8px;
450         margin-right: 229px;
451         padding-top: 5px;
452         padding-bottom: 5px;
453     }
454 }
455 }
456 }
457 }
458 }
```

观察发现，TRAE Figma 生成的 tsx 和 Figma MCP 提供的并不是完全一样的，可能是自行实现的解析（不确定）。

此外，这种方式目前没有支持 Code connect，相当于这部分需要模型自行发觉或者做出额外的指引。

4.3 自行生成 IR

这是一种高阶玩法，赋予了团队最大的灵活性和控制力，但同时也带来了最高的心智负担。

工作原理：不依赖任何现有工具，通过直接调用 Figma 的 REST API，获取设计稿的完整 JSON 数据。然后，编写自定义脚本，将这些原始数据解析、清洗、并转换为一种标准化的、AI 更易于理解的**中间表示（Intermediate Representation, IR）**。

核心挑战：Figma API 返回的原始数据极为庞大且复杂。如何设计一套高效的 IR 格式，并编写脚本做好剪枝和转换，对开发者的要求非常高。如果 IR 设计不佳，过大的中间产物反而会严重干扰模型的注意力。

💡 此方法适用于有充足研发资源、且对 D2C 流程有深度定制化需求的团队。对于大多数团队而言，投入产出比不如方法一。

Figma OpenAPI: <https://developers.figma.com/docs/rest-api/file-endpoints/#get-file-nodes-endpoint>

如同上文提到的 [Figma-Context-MCP](#) 所做的一样，我们可以通过 Figma 的官方 OpenAPI 获取设计稿的完整信息，根据需要来定制化的将设计稿转换成我们想要提供给模型的信息。

如下是一个带有辅助脚本的 SKILL，来自 TRAE Solo：

代码块

1

交互流程

2

3

用户提供 Figma URL + 交互需求

4

↓

5

阶段一：获取 Figma 数据（运行脚本）

6

↓

7

阶段二：裁剪组件 node.json（暂时通过 AI，提取目标组件）

8

↓

9

阶段三：生成交互 IR（定义 Props、Events、States）

10

↓

11

阶段四：用户确认交互规范

12

↓

13

阶段五：生成代码（React + CSS Module）



figma-to-code.zip

32.85KB





 SKILL.md

它会按一定的映射规则，处理 Figma 接口返回的信息

Figma 属性	IR 属性	说明
<code>layoutMode: HORIZONTAL</code>	<code>layout.direction: 'row'</code>	水平布局
<code>layoutMode: VERTICAL</code>	<code>layout.direction: 'column'</code>	垂直布局
<code>primaryAxisAlignItems</code>	<code>layout.justify</code>	主轴对齐
<code>counterAxisAlignItems</code>	<code>layout.align</code>	交叉轴对齐
<code>itemSpacing</code>	<code>layout.gap</code>	元素间距
<code>fills[].type: SOLID</code>	<code>styles.background[].type: 'solid'</code>	纯色背景
<code>strokes + strokeWidth</code>	<code>styles.border</code>	边框
<code>cornerRadius</code>	<code>styles.borderRadius</code>	圆角
<code>effects[].type: DROP_SHADOW</code>	<code>styles.shadow</code>	阴影
<code>type: TEXT</code>	<code>type: 'text' + meta.textContent</code>	文本节点

运行后，即可获取 Figma IR JSON。

实战对比

为了直观地检验上述三种方法的效果，我们使用同一个 Figma 设计稿，分别在 **Claude Opus**、**Gemini 3 Pro** 和 **GPT-5.2-Codex** 三个主流模型上进行了实战测试。

核心结论速览

方法	最佳模型表现 (Claude Opus)	主要问题	综合评价
Figma Local MCP + Code Connect	布局、间距、组件引用均准确，仅需微调文案	Code Connect 会丢失组件内文案。	效果最佳。 虽然有小瑕疵，但修改成本远低于重构，已

	和少数样式。	错误的组件映射会误导 AI。	具备生产力价值。
TRAE Figma	文案正确，但所有元素均为重新生成的 div ，未复用组件。	无工程化概念	便捷但不能说质量高。 适用于快速生成静态页面初稿，但难以直接用于生产。但也可能有惊喜
自定义 IR 脚本	大体框架正确，但布局和组件位置有误。	- 中间产物过大，AI 注意力涣散。 - 缺少 Assets 和 Design Token。	有潜力但门槛高。 当前效果不佳，但通过优化 IR 剪枝逻辑，有望提升。

目标

以上次分享的 case 为例

访问控制

公网

私网

若需要通过私网访问 TRAE 企业专属版，可按照如下步骤进行操作

了解更多，可查看[配置详情](#)

1 创建私有网络终端节点

前往私有网络产品创建终端节点

配置参数指引

查看示例

添加方式

火山引擎服务

终端节点服务

com.xxx.xxx

私有 DNS 名称

启用

完成购买

去创建终端节点

2 检查可用状态

购买完成后，根据列表状态判断是否可用

检查列表，满足以下条件则表示【私有网络可用】

实例状态

可用

连接状态

已连接

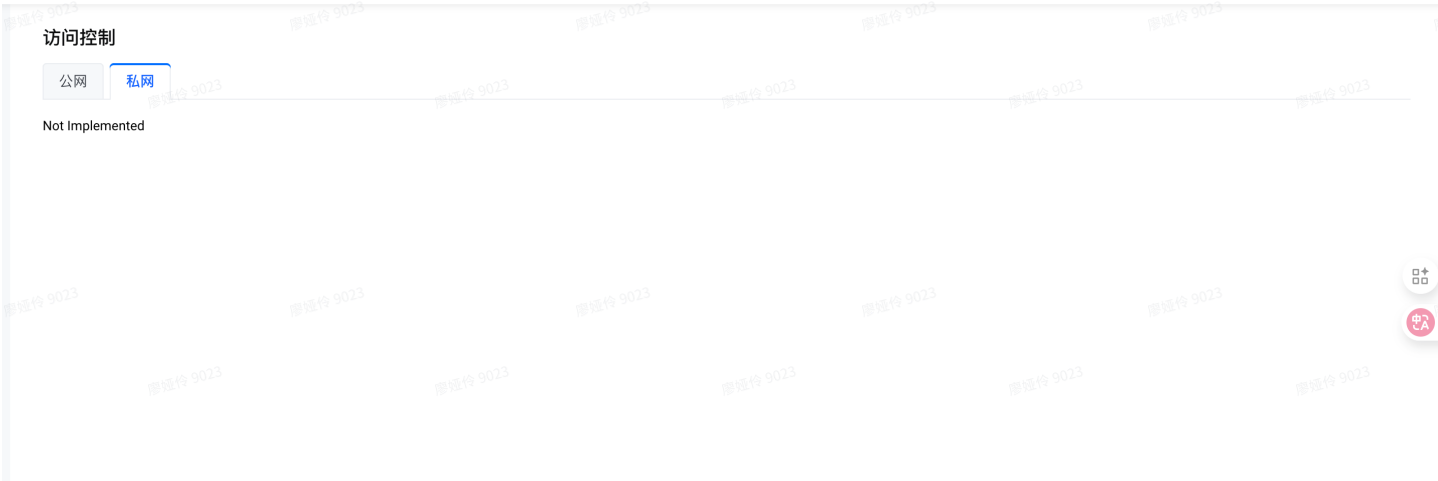
实际需要结构

```
1 <TabPane key="private" title="私网">
2   <div className={cx('private-wrap')}>
3     <div className={cx('private-header')}>
4       <h3>
5         若需要通过私网访问 TRAE {instanceLabel}，可按照下步骤进行操作
6       </h3>
```

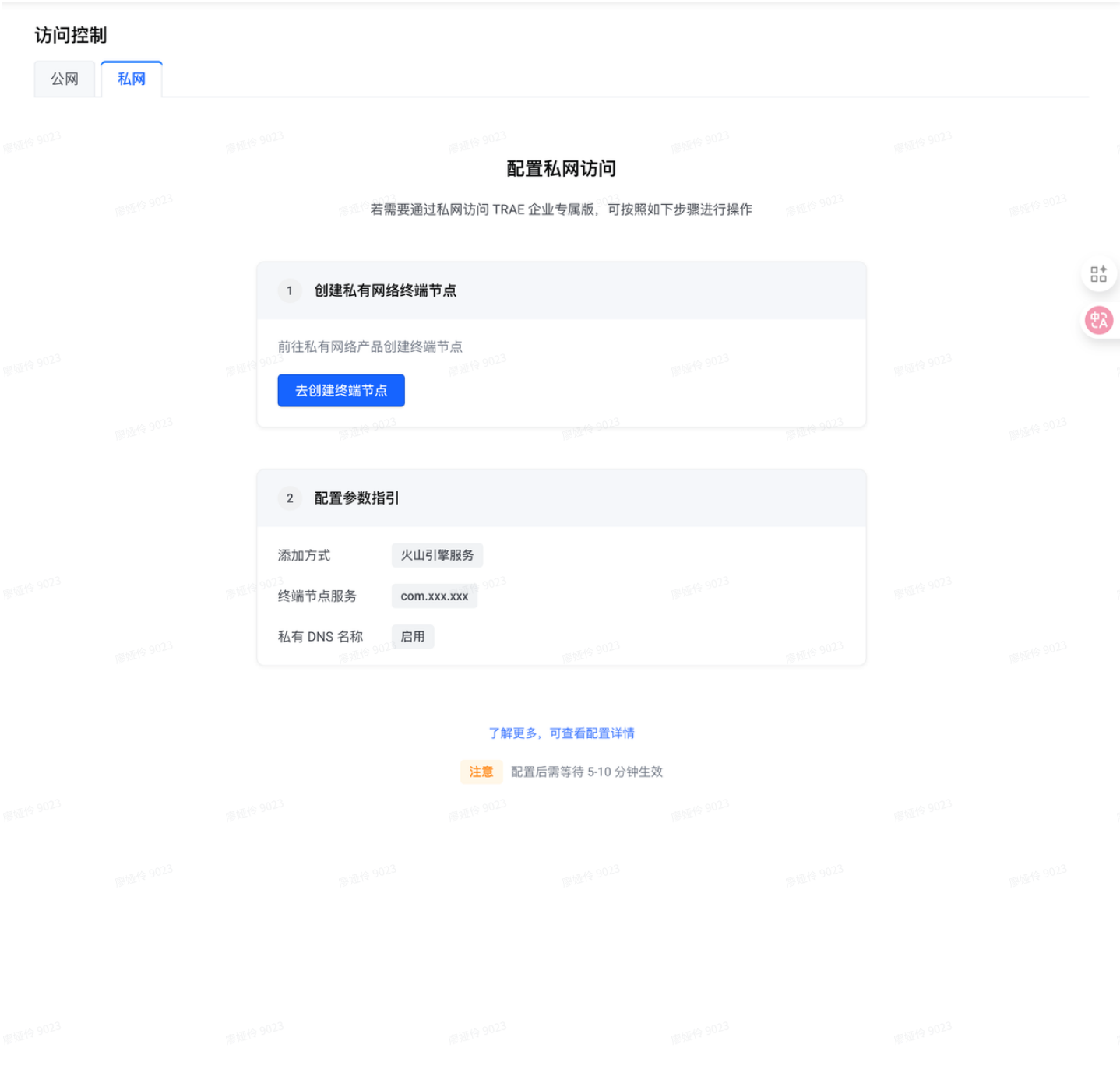
```
7 <div className={cx('sub-actions')}>
8   了解更多，可查看
9   <a
10     href={EXTERNAL_LINK.PRIVATE_NETWORK_CONF_DETAIL}
11     target="_blank"
12     rel="noreferrer noopener"
13   >
14     配置详情
15   </a>
16 </div>
17 </div>
18
19 <div className={cx('private-steps')}>
20   <div className={cx('step-card')}>
21     <div className={cx('step-head')}>
22       <div className={cx('step-head-left')}>
23         <div className={cx('step-index')}>1</div>
24       <div>
25         <div className={cx('step-title')}>
26           创建私有网络终端节点
27         </div>
28         <div className={cx('step-sub')}>
29           前往私有网络产品创建终端节点
30         </div>
31       </div>
32     </div>
33     <Button
34       type="primary"
35       onClick={() =>
36         consoleDomain &&
37         window.open(
38           VOLC_LINK.ACCESS_CONTROL_PRIVATE_NETWORK_CONF,
39           '_blank',
40         )
41       }
42       disabled={!consoleDomain}
43     >
44       去创建终端节点
45     </Button>
46   </div>
47
48   <div className={cx('params')}>
49     <div className={cx('params-head')}>
50       <span>配置参数指引</span>
51     <Popover
52       trigger="hover"
53       content={
```

```
54         <div className={cx('example-pop')}>
55             <img
56                 src={picPrivate1}
57                 className="h-[108px]"
58                 alt="示例1"
59             />
60             <img
61                 src={picPrivate2}
62                 className="h-[61px]"
63                 alt="示例2"
64             />
65             <div>可参照实例，完成 [创建终端节点] 配置</div>
66         </div>
67     }
68 >
69     <span className={cx('example-link')}>查看示例</span>
70 </Popover>
71 </div>
72 <div className={cx('param-row')}>
73     <LinePrefixMark className={cx('prefix-mark')} />
74     <span className={cx('param-label')}>添加方式</span>
75     <Tag size="small" bordered>
76         火山引擎服务
77     </Tag>
78 </div>
79 <div className={cx('param-row')}>
80     <LinePrefixMark className={cx('prefix-mark')} />
81     <span className={cx('param-label')}>终端节点服务</span>
82     <Tag size="small" bordered>
83         {VPC_CONFIG.PRIVATE_NETWORK_ENDPOINT}
84     </Tag>
85 </div>
86 <div className={cx('param-row')}>
87     <LinePrefixMark className={cx('prefix-mark')} />
88     <span className={cx('param-label')}>私有 DNS 名称</span>
89     <Tag size="small" bordered>
90         启用
91     </Tag>
92 </div>
93 <div className={cx('param-row')}>
94     <LinePrefixMark className={cx('prefix-mark')} />
95     <span className={cx('param-label')}>完成购买</span>
96 </div>
97 </div>
98 </div>
99
100 <div className={cx('step-card')}>
```

```
101 <div className={cx('step-head')}>
102   <div className={cx('step-head-left')}>
103     <div className={cx('step-index')}>2</div>
104     <div>
105       <div className={cx('step-title')}>检查可用状态</div>
106       <div className={cx('step-sub')}>
107         购买完成后，根据列表状态判断是否可用
108       </div>
109     </div>
110   </div>
111 </div>
112
113 <div className={cx('params')}>
114   <div className={cx('params-head')}>
115     检查列表，满足以下条件则表示【私有网络可用】
116   </div>
117   <div className={cx('param-row')}>
118     <LinePrefixMark className={cx('prefix-mark')} />
119     <span className={cx('param-label')}>实例状态</span>
120     <Tag size="small" bordered>
121       <IconCheckCircleFill /> 可用
122     </Tag>
123   </div>
124   <div className={cx('param-row')}>
125     <LinePrefixMark className={cx('prefix-mark')} />
126     <span className={cx('param-label')}>连接状态</span>
127     <Tag size="small" bordered>
128       <IconCheckCircleFill />
129       已连接
130     </Tag>
131   </div>
132 </div>
133 </div>
134 </div>
135 </div>
136 </TabPane>
```



TRAE 市场默认 Figma MCP



通过私网访问企业专属版

推荐 通过配置私有网络（PrivateLink），在 VPC 内安全访问 TRAE 企业专属版控制台。

去私网配置控制台

查看配置文档

服务名称 **复制**

在私有网络控制台创建终端节点服务时，选择该服务名称。

访问域名 **复制**

在本地网络解析中，将终端节点域名解析到该访问域名，实现通过私网访问控制台。

- 使用说明**
1. 前往私有网络控制台，根据上述服务名称创建终端节点服务。

2. 在终端节点配置中，绑定需要访问 TRAE 企业专属版的 VPC。

3. 在 VPC 内配置 DNS，将终端节点域名解析到访问域名。

4. 在绑定的 VPC 内，通过浏览器访问上述访问域名即可进入企业专属版。

Figma Local MCP with SKILL & Code Connect

/implement-design 参照提供的设计稿实现私网 tab 页面

过程

> 参考了 1 个上下文

调用技能: implement-design

现在我将按照 implement-design 技能的工作流程来实现这个任务。让我首先解析 Figma URL 并获取设计上下文。

步骤 1: 解析 Figma URL

从 URL

• File key:

• Node ID:

让我获取设计上下文和截图，同时查看现有的组件代码：

figma-desktop/get_design_context

figma-desktop/get_screenshot

src/components/access-control/index.tsx

src/components/access-control/styles/index.module.less

我已获取到设计上下文和截图。让我创建一个任务列表来跟踪实现进度，并下载所需的 SVG 资源：

✓ 下载 Figma 设计中的 SVG 资源

✓ 分析设计上下文和现有代码结构

✓ 实现私网 Tab 页面的 UI 组件 & 添加样式到 index.module....

✓ 验证实现与 Figma 设计的一致性

我已经完成了私网 Tab 页面的。以下是实现的总结：

修改的文件

1. src/assets/line-prefix-mark.svg - 新建

• 创建了配置项前缀标记的 SVG 图标

2. src/components/access-control/index.tsx - 修改

• 新增 PrivateNetworkTab 组件，包含：

• 页面标题和帮助文档链接

• 两个步骤卡片（创建私有网络终端节点、检查可用状态）

• 配置参数指引（添加方式、终端节点服务、私有 DNS 名称、完成购买）

• 状态检查列表（实例状态、连接状态）

• 配置示例弹窗

• 将私网 Tab 内容从 "Not Implemented" 替换为 PrivateNetworkTab 组件

3. src/components/access-control/styles/index.module.less - 修改

• 添加了私网页面所需的所有样式类

设计实现要点

根据 Figma 设计，实现了以下 UI 元素：

• 页面布局：居中显示，带有标题和帮助链接

• 步骤卡片：两个并排的卡片，带有阴影和圆角边框

• 步骤编号：圆形灰色背景的数字标记

• 配置项列表：带有蓝色/灰色前缀标记的配置项

• Tag 标签：蓝色标签显示配置值，绿色标签显示成功状态

• 弹窗：点击"查看示例"显示配置示例图片

功能点

• 点击"去创建"按钮跳转到火山引擎私有网络终端节点创建页

结果

• Claude Opus



Claude Opus 实现的和我们期望的大差不差，差别主要有 prefix-mark 颜色，按钮的文案不对，以及右侧 Tag 样式及文案不对。

但这也并非模型的问题，分析发现这是 Code Connect 导致的。

1. Code Connect 会丢失掉组件内部的文案，相当于“去创建终端节点”、“已连接”的文案压根就没有被 MCP 提供给模型。
2. 错误的 Code Connect 也会带来麻烦。“已连接”Tag在组件库内是源力组件二次封装的 `StatusTag` ,但在 Figma 里连接的是 `Tag` ,对模型造成了误导，需要进行手动修改

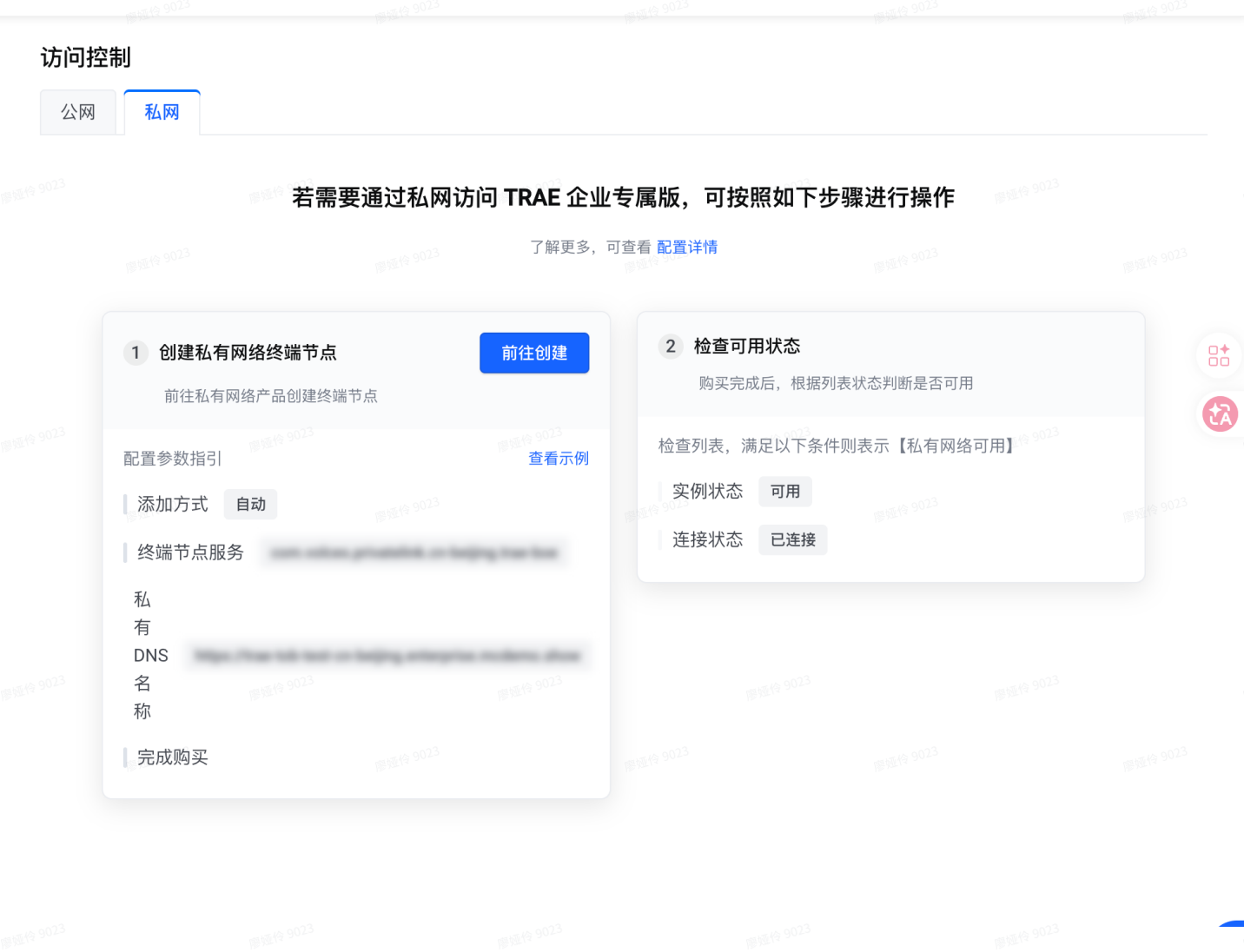
• Gemini 3



Gemini 在上文提到的问题都有的情况下，额外出现了布局方面没有识别出两个卡片需要对齐，按钮也出现了错位。总体体验劣于 Claude Opus。

- GPT-5.2-Codex

慢！很慢！因为他真的去找仓库的各种代码细节尝试理解并尝试实现出完整的代码功能了。但是慢并没有带来更好的效果，反而因为获取太多信息注意力涣散了，最终的效果并不好。



总结

总体来说 Claude>Gemini>GPT。Claude 提供的产物里，字体字号，间距大体都是对的，需要人工介入修改的部分不是很多，已经可以接受了。

尽管 Code Connect 会丢失文案，但是修改文案的成本要比修改组件引用的成本小的多，综合体验下来还是推荐增加 Code Connect 关联的。

优点

- 支持 Code Connect，给模型和代码仓库更强的相关性
- 可以直接从本地服务器获得图片、Icon 等 Asset，确保素材引用的正确性，而非自己胡编乱造。
- Figma MCP 里自带约束 Prompt

- 适配性强

缺点

- Code Connect 会丢失组件内部文案，需要取舍是否进行添加
- 需要本地运行 Figma App，占用内存资源（Remote 模式看起来 Trae 还没支持）
 - Figma App 还得打开对应的页面，不可以APP 看 A 设计稿的时候让 AI 实现 B 设计稿，否则 MCP 不返回内容。
 - 开发机难以使用（SSH 反代会很麻烦，很容易断连）

TRAE Figma

Frame 2119903758

screenshot_7218_16615.png

{{设计稿 url}}

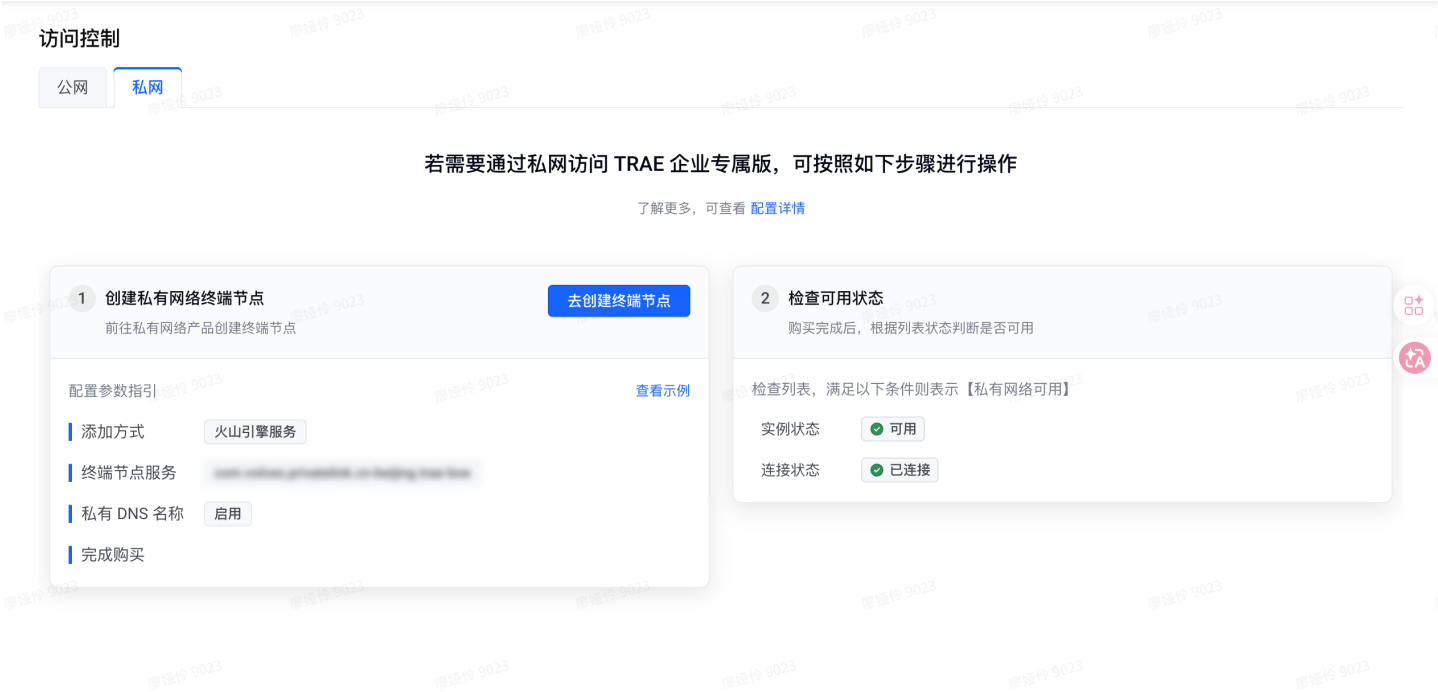
为 src/components/access-control/index.tsx 参照提供的设计稿实现私网 tab 页面

Claude Opus



使用 TRAE Figma 后，模型对文案的理解都是正确的， 但是没有了Code Connect，所有的元素都是他自己造的，没有关联到代码仓库中的组件，这带来了更高的维护成本。素材方面，即使TRAE 在.figma/images里提供了相关的 svg，但是模型还是选择了自己看图生成，并不符合要求。

Gemini 3 Pro



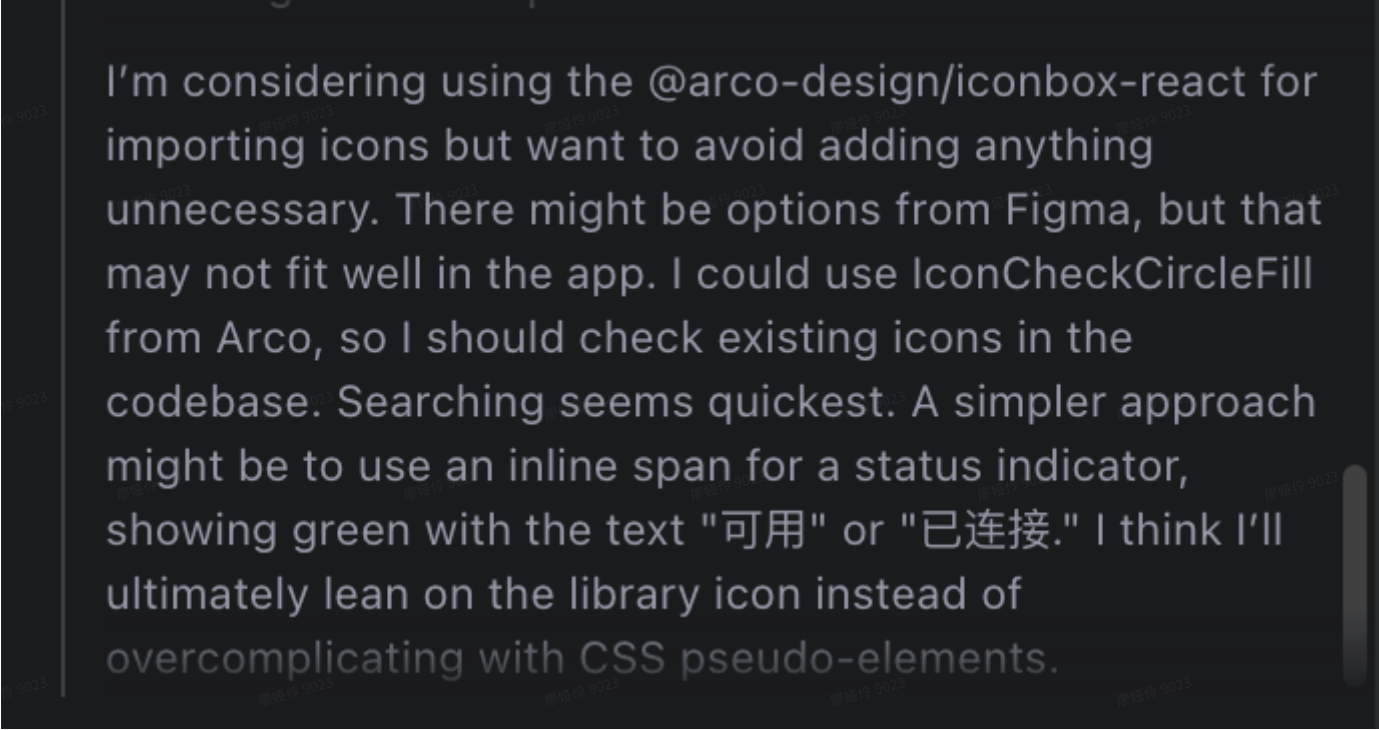
自作聪明添加了参数指引的定宽对齐，卡片也没有定宽定高。
也是 Claude 实现的比 Gemini3 精准。

GPT 5.2-Codex



TRAE Figma + GPT 5.2 竟然实现出了其他方式都没实现出的 对号 Icon？就在我惊喜不已的时候，仔细观察却发现空欢喜一场：GPT5.2 只是觉得这里有个 icon，然后根据项目联想到了 Arco Design 组件库的 IconCircleFill，而不是从上下文中检索出来的。

序号、prefixmark 的实现是失准的，也证明了它其实并没有能很好地关联到 TRAE Figma 下载到的 svg。



I'm considering using the @arco-design/iconbox-react for importing icons but want to avoid adding anything unnecessary. There might be options from Figma, but that may not fit well in the app. I could use IconCheckCircleFill from Arco, so I should check existing icons in the codebase. Searching seems quickest. A simpler approach might be to use an inline span for a status indicator, showing green with the text "可用" or "已连接." I think I'll ultimately lean on the library icon instead of overcomplicating with CSS pseudo-elements.

总结

这种方式可能需要更进一步定制化地撰写一个 SKILL 约束其行为，来在组件代码关联、使用下载的 assets 等方面获得更好的效果。但不支持 Code Connect 仍然是一个较为深切的痛点——万一模型没有正确关联组件关系，改组件的 props 可比从 div 换成组件简单的多。

优点

- TRAE IDE 原生结合，不需要单开 Figma App
- 直接提供设计稿区域截图和生成文件，不需要额外调用 MCP 占用上下文

缺点

- 不支持 Figma Code Connect
- 无 Design Tokens 映射
- 生成产物要比 Figma MCP 大

自行生成IR

| /figma-to-code 为 src/components/access-control/index.tsx 参照提供的设计稿实现私网 tab 页面

Claude Opus

过程

/figma-to-code 为
src/components/access-control/index.tsx
参照 ` <https://www.figma.com/des...> ` 实
现私网 tab 页面

SOLO Coder

> 参考了 1 个上下文

- ✓ 分析 Figma 设计稿结构 >
- ✓ 添加私网 tab 页面的样式 >
- ✓ 实现私网 tab 页面组件 >
- ✓ 验证代码并运行 lint 检查 >

我已经根据 Figma 设计稿为 `src/components/access-control/index.tsx` 实现了私网 tab 页面。以下是实现的内容：

实现概述

1. 页面结构

根据 Figma 设计稿，私网 tab 页面包含：

- 头部介绍区域
 - 主标题："若需要通过私网访问 TRAE 企业专属版，可按照以下步骤进行操作"
 - 副标题带链接："了解更多，可查看配置详情"
- 两个步骤卡片
 - 步骤 1：创建私有网络终端节点
 - 配置参数指引（添加方式、终端节点服务、私有 DNS 名称、完成购买）
 - "创建终端节点" 按钮

结果

• 两个步骤卡片

◦ 步骤 1：创建私有网络终端节点

- 配置参数指引（添加方式、终端节点服务、私有 DNS 名称、完成购买）
- "创建终端节点" 按钮

◦ 步骤 2：检查可用状态

- 检查列表（实例状态、连接状态）

2. 修改的文件

- `index.tsx` - 添加了私网 tab 的 JSX 结构
- `index.module.less` - 添加了私网 tab 的所有样式

3. 样式特点

- 卡片带有阴影和圆角边框
- 步骤编号使用圆形背景
- 配置项使用左侧指示条
- 状态标签使用灰色背景和边框
- 整体布局居中对齐

产物汇总

</> 2个文件已更改 +326 -1

查看变更

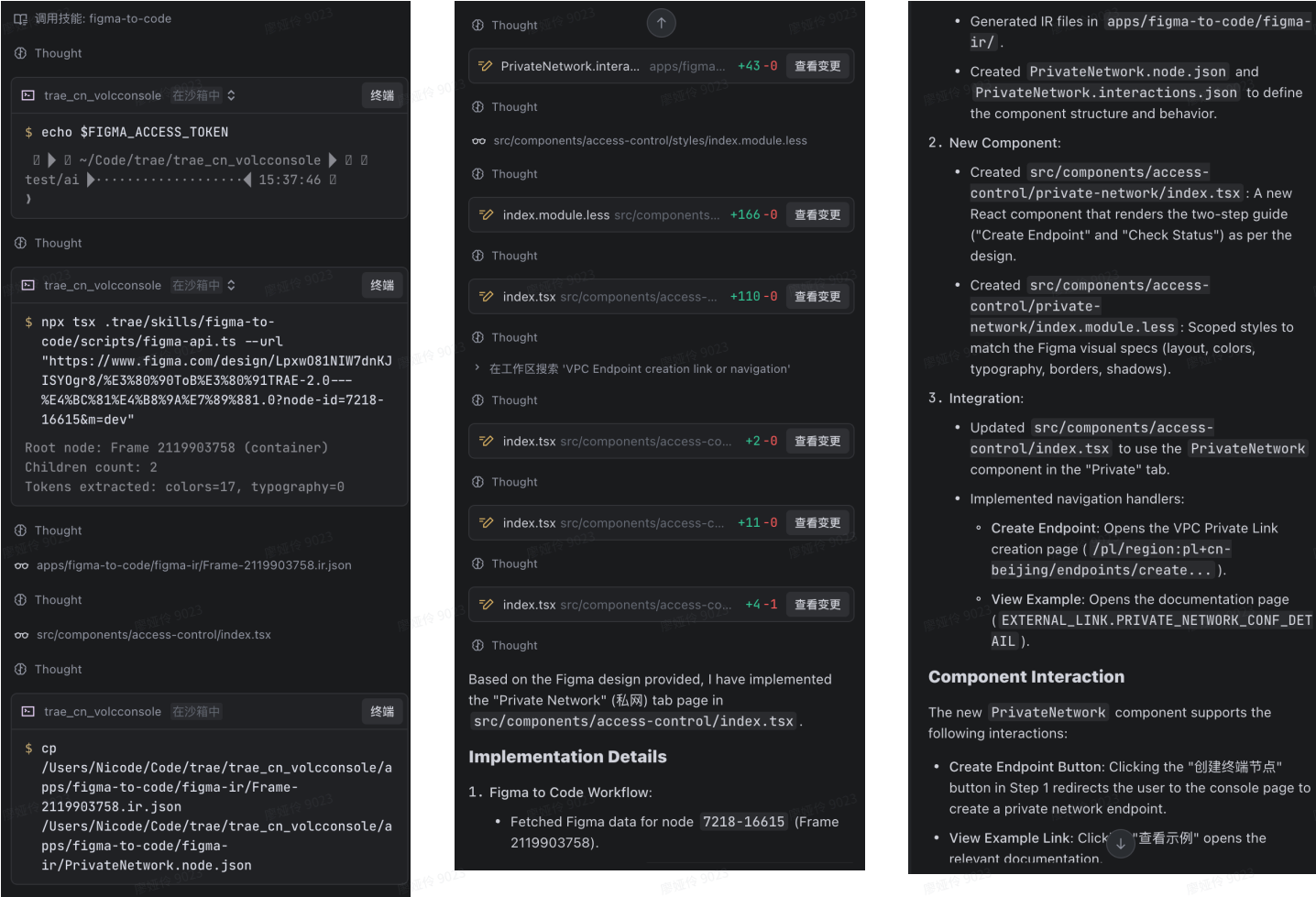


缺少 Assets，DesignToken，大体框架正确，但是按钮的位置错误。

而且过程中并没有遵照 Skill 去进行裁剪 node.json和 interact.json,可能是过大的 IR JSON 已经开始让模型注意力涣散了。

Gemini 3

过程



结果



基础布局正确，但 tag，button，assets还是不对，仍然是硬伤

GPT5.2-Codex

trae_cn_volcconsole 在沙箱中

终端

```
$ npx tsx .trae/skills/figma-to-code/scripts/figma-api.ts --url "https://www.figma.com/design/Lpxw081NIW7dnKJISY0gr8/%E3%80%90ToB%E3%80%91TRAE-2.0---%E4%BC%81%E4%B8%9A%E7%89%881.0?node-id=7218-16615&m=dev"
```

Root node: Frame 2119903758 (container)

Children count: 2

Tokens extracted: colors=17, typography=0

apps/figma-to-code/figma-ir/Frame-2119903758.ir.json

Thought

src/components/access-control/index.tsx

Thought

src/components/access-control/styles/index.module.less

Thought

✓ 实现私网 Tab 页面结构与内容 & 补充私网 Tab 页面... >

src/components/access-control/index.tsx

Thought

完成内容

- 实现私网 Tab 页面结构与交互链接（含步骤卡片、按钮与示例链接）：`index.tsx:L242-L450`
- 补充私网 Tab 对应样式（布局、卡片、标签与引导条）：`index.module.less:L125-L305`

结果

访问控制

公网 私网

若需要通过私网访问 TRAE 企业专属版，可按照如下步骤进行操作

了解更多，可查看配置详情

1 创建私有网络终端节点

前往私有网络产品创建终端节点

前往创建

配置参数指引

查看示例

添加方式 必填

终端节点服务 必填

私有 DNS 名称 必填

完成购买

2 检查可用状态

购买完成后，根据列表状态判断是否可用

检查列表，满足以下条件则表示【私有网络可用】

实例状态 运行中

连接状态 已连接

tag 和 prefix mark 不对、查看详情链接丢失

总结

这种方式生成的中间产物最大，所以模型的注意力会被严重牵扯。

I'm thinking through the process of delivering the skill, and while there are requirements, I want to clarify if every step is truly necessary. I realize the tree size is significant, around 2500 lines, which could be too much output. The user specifically asked to use "figma-to-code," so I need to follow that. I plan to create a node.json file, extracting from the IR tree, but I wonder if I can keep it more concise.

↑不堪重负的 GPT5.2,已经不觉得继续裁剪 node.json会让其更“精确”

但因为是脚本，所以生成产物的大小是可以通过完善脚本的剪枝逻辑优化的。比如现在 IR json 是交给 Chat 模型自行剪枝，如果能通过逻辑/额外模型做好剪枝，交付给主模型的是 `interaction.json`，估计效果还能优化一些。

优点

- 自主控制逻辑，可以实现定制化优化

缺点

- 中间产物过大（目前）
- 需要自行实现/优化逻辑，需要的心智负担较大
- 项目代码中会驻留中间产物
- 不支持 Design token，无法获得 Asset

总结

	工具提供给模型的中间产物
Figma Local MCP	• 181 行 Tailwind 风格 JSX （其中包含 Code Connect 映射关系） • Design Token 映射关系
TRAE Figma	• 127 行 JSX • 458 行LESS 样式文件 • 一张截图 • assets
自行解析	• 2574行的 IR JSON （需要 AI 裁剪）

从目前的使用体验来看，使用 Figma Local MCP，配合 Code Connect 和 SKILL 的使用体验是最好的。

- 先期准备：Review 设计稿质量、做好 Code Connect
- 开发：Figma Local MCP
- 模型推荐：Claude Opus > Gemini 3 Pro Preview / GPT 5.2-Codex （后两者互有胜负）

场景 / 诉求	🏆 首选方案	🥈 备选方案
追求最高工程质量与保真度	Figma Local MCP + Code Connect 这是唯一能实现组件级复用的方法，代码质量最高。	自定义 IR 脚本 通过深度定制，有潜力实现高质量输出，但需要巨大前期投入。
在开发机/云端 IDE 工作	TRAE Figma 便捷性最高，无需本地环境。适合快速验证 UI 或生成静态页面。	自定义 IR 脚本 如果脚本部署在云端，也可以实现远程工作流。
快速搭建原型，不关心代码质量	TRAE Figma 操作最简单，“一键”即可获得视觉上相似的初稿。	无

★ 最终推荐工作流 (本地开发):

- 准备阶段:** 设计师与工程师共同审查设计稿，确保其遵循设计规范，并完成关键组件的 **Code Connect** 映射。
- 开发阶段:** 工程师在本地启动 Figma App 和 Figma MCP Server。
- 执行阶段:** 采用**模块化拆分策略**，使用搭载了 **Claude Opus** 模型的 AI，通过 **SKILL 逐个模块**生成代码。
- 调优阶段:** 工程师审查 AI 生成的代码，手动修正文案、样式等细节，并集成业务逻辑。

未来展望：实现 AI 的“自调试”闭环

当前的 D2C 工作流仍然是单向的：AI 生成代码后，它的任务就结束了。但一个更理想的未来是，AI 能够“看到”自己所写代码的实际渲染效果，并与原始设计稿进行比对，从而实现**自我调试和修正**。

这其中蕴含着几个值得探索的技术方向：

- 视觉回归测试:** 我们可以尝试使用 Playwright、Cypress 等工具，在 AI 生成代码后自动启动开发服务器，并对渲染出的组件进行截图。
- AI Agent 的自我修正:** 将组件截图再次输入给一个多模态 AI Agent，让它扮演“视觉 QA”的角色，判断截图与原始设计稿的差异，并生成新的指令来修正代码。
- IDE 内的视觉反馈:** TRAE 正在探索的 Visual Editor 可以直接在编辑器中预览组件渲染效果，并允许开发者框选视觉差异、生成反馈指令，形成一个在 IDE 内部的快速迭代闭环。



微信搜一搜



TRAE企业版